Identifying a Rigid Body's Moment of Inertia Matrix with a Two-Stage Physics-Informed Neural Network

Author: Nói Halldórsson

Supervisor: Vakhtang Putkaradze

Course: MA PH 499

Course Instructor: Don Page

University of Alberta

2025-12-2

Abstract

We study the inverse problem of identifying a rigid body's moment of inertia from angular velocity measurements by formulating a two-stage physics-informed neural network (PINN) for the Euler rigid body equation under a sequence of delta-like torque impulses. In Stage 1, a fully connected neural network is trained purely on synthetic angular velocity data generated from a known inertia tensor and impulsive torque profile, yielding a smooth approximation of the trajectory $\mathbf{\Omega}(t)$ over [0,1]. In Stage 2, the network weights from Stage 1 are frozen, the Euler rigid body equation is enforced through a physics-based loss, and the six independent components of the inertia tensor are treated as trainable parameters. This decoupled strategy allows the PINN to refine only the inertia parameters while remaining consistent with the learned trajectory. Numerically, Stage 1 attains a data loss of 5.5×10^{-8}, and Stage 2 converges to an inertia tensor that closely matches the ground-truth values with a final physics loss of 6.2×10^{-4}. We further benchmark this hand-designed two-stage PINN against a PINN implementation generated by the ChatGPT 5.1 Thinking model, which exhibits noisy training, larger final loss 3.8×10^{-1}, and poor parameter recovery. The results highlight the effectiveness of two-stage PINNs for inertia identification and illustrate both the promise and current limitations of

large language models as assistants for specialized physics-informed machine learning workflows.

1 Introduction

Accurate modeling of rigid body rotational dynamics dynamics is fundamental to many applications such as spacecraft attitude, robotics, and mechanical systems designs. In these settings the angular velocity of the body evolves according to the Euler rigid body equation which couples the angular velocity with the moment of inertia tensor. The inertia tensor encodes how mass is distributed inside the body and directly affects rotational stability, control effort, and energy consumption. However, in many practical scenarios the inertia parameters are often unknown or known approximately which motivates the inverse problem of estimating the moment of inertia from observed angular motion.

Classical approaches to inertia identification typically combine analytical rigid body models with parameter estimation techniques such as least squares or recursive estimation using measurements of torque, attitude, and angular velocity. Recent work in determining the inertia tensor have explored physics based learning schemes that embed the rigid body equations directly into analytical modeling algorithms. For instance, Mammarella et al. proposes a blended physics based identification framework that that estimates a spacecrafts inertia tensor by integrating the physical constraints and neural network like components to a unified optimization procedure [1]. Such methods highlight the the benefits of combining physical laws and data driven models when identifying physical parameters.

Physics Informed Neural Networks (PINNs) have become a powerful framework for solving forward and inverse problems governed by differential equations. Raissi et al. introduced PINNs as neural networks trained not only on observational data but on the residuals of the governing equations, forcing the network to obey conservation laws and boundary conditions [2]. PINNs have since been applied to many industries such as robotics and structural engineering [3] [4]. For example, Guo and Fang developed a physics informed framework to to determine structural parameters where physical constraints were incorporated to guide the network toward mechanically consistent solutions [4].

Despite the progress in the use of PINNs there is comparatively little work that applies PINNS directly to the Euler rigid body equation with the purpose of determining the moment of inertia from angular velocity. Therefore, we address this gap by formulating a two stage PINN to determine to determine the moment of inertia tensor from synthetic angular velocity data. In Stage 1 we train a standard forward feeding neural network purely from data to approximate the angular velocity vector $\Omega(t)$, obtaining a smooth substitute of the synthetic rotational motion without imposing physical constraints. In Stage 2 we freeze the neuron weights of the Stage 1 layers and, introduce the Euler rigid body equation into the loss function, and make the moment of inertia components trainable parameters for the network. Thus, Stage 2 becomes a PINN-based inverse problem in which the network is constrained to produce an inertia tensor from the rigid body dynamics that is consistent with the trajectory of Stage 1.

Lastly, this work also examines the role of large language models (LLM) as programming assistants for machine learning. LLMs such as ChatGPT are increasingly used to generate and refine code for numerical simulation, but there remain questions about its efficency and reliability especially when it comes to specialized tasks like physics informed training networks. Therefore, we implement the proposed PINN problem to the ChatGPT 5.1 Thinking model and compare with my algorithm the computational efficiency, convergence behavior, and identified parameters.

2 Theory

2.1 Rigid body dynamics

Given that for a rigid body the moment of inertia tensor $\mathbf{I}$ is not constant in the spatial frame but depends upon the orientation matrix we choose to work in the body frame where the tensor $\mathbf{I}$ is constant. With that consider a rigid body with body fixed frame with angular position and velocity vectors

$$\mathbf{\Omega}(t) = (\omega_1, \omega_2, \omega_3)^T \, , \, \dot{\mathbf{\Omega}}(t) = (\dot{\omega}_1, \dot{\omega}_2, \dot{\omega}_3)^T$$

expressed on an arbitrary axis. In this frame the moment of inertia tensor has the arbitrary form

$$\mathbf{I} = \begin{pmatrix} I_{11} & I_{21} & I_{31} \\ I_{12} & I_{22} & I_{32} \\ I_{13} & I_{23} & I_{33} \end{pmatrix}$$

where $I_{11},\ I_{22}, I_{33} > 0$ and by symmetry $I_{12} = I_{21}$,$I_{13} = I_{31}$,$I_{32} = I_{23}$.This inertia tensor $\mathbf{I}$ about the centre of mass is a real, symmetric, positive definite 3x3 matrix. Symmetry follows from the definition of the products of inertia and positive definitness follows from the expression of rotational kinetic energy $T = \frac{1}{2}\boldsymbol{\Omega}^T\mathbf{I}\boldsymbol{\Omega}$ which is strictly positive for non-zero angular velocity $\boldsymbol{\Omega}$. The rotational dynamics are governed by the Euler rigid body equation

$$\mathbf{I}\dot{\boldsymbol{\Omega}}(t) + \boldsymbol{\Omega}(t) \times (\mathbf{I}\boldsymbol{\Omega}(t)) = \boldsymbol{\tau}(t) \tag{1}$$

where $\boldsymbol{\tau}(t) = [\tau_1(t), \tau_2(t), \tau_3(t)]^T$ denotes the external torque expressed in the same body fixed frame. In this project we assume that the angular velocity $\boldsymbol{\Omega}(t)$ is available for a finite set of measurement times $\{t_n\}_{n=1}^N$ and that the torque $\boldsymbol{\tau}(t)$ is known. The moment of inertia parameters $I_{11}, I_{22}, I_{33}, I_{21}, I_{31}, I_{32}$ are treated as the unknown parameters. We seek inertia parameters that minimize the difference in the left and right hand sides of (1) when evaluated along the observed motion.

2.2 Physics informed neural network formulation

For the problem above we employ a physics informed neural network (PINN). Let

$$\hat{\boldsymbol{\Omega}}_\theta(t) \approx \boldsymbol{\Omega}(t)$$

denote a neural network parametrized by weights contained in θ. The network takes time t as an input and outputs an approximation of the angular velocity vector using the weights. To do this we introduce a data residual given by

$$\mathbf{R}_{\text{data}}(t;\theta) = \hat{\boldsymbol{\Omega}}_\theta(t) - \boldsymbol{\Omega}_{\text{data}}(t) \tag{2}$$

In an ideal model $\mathbf{R}_{\text{data}}(t;\theta) \equiv 0$ for all t, however, in practice we enforce small residuals at a set of collocation points $\{t_m\}_{m=1}^{M}$ by minimizing the data based loss

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{M}\sum_{m=1}^{M}\left\|\hat{\mathbf{\Omega}}_\theta(t_m) - \mathbf{\Omega}(t_m)\right\|^2 \tag{3}$$

This loss function measures how well the network fits the observed angular velocity. Next, we define the physics residual associated with Euler's equation as

$$\mathbf{R}(t;\phi) = \mathbf{I}_\phi\dot{\hat{\mathbf{\Omega}}}_\theta(t) + \hat{\mathbf{\Omega}}_\theta(t) \times \left(\mathbf{I}_\phi\hat{\mathbf{\Omega}}_\theta(t)\right) - \boldsymbol{\tau}(t) \tag{4}$$

where again we enforce small residual at the same collocation points $\{t_m\}_{m=1}^{M}$ by minimizing the physics based loss

$$\mathcal{L}_{\text{phys}}(\phi) = \frac{1}{M}\sum_{m=1}^{M}\left\|\mathbf{I}_\phi\dot{\hat{\mathbf{\Omega}}}_\theta(t) + \hat{\mathbf{\Omega}}_\theta(t) \times \left(\mathbf{I}_\phi\hat{\mathbf{\Omega}}_\theta(t)\right) - \boldsymbol{\tau}(t)\right\|^2 \tag{5}$$

2.3 Two stage PINN parameter identification

In this work, we use a two stage training strategy that separates learning the angular velocity trajectory and the moment of inertia parameters.

2.3.1 Stage 1: data-driven trajectory learning

Here the inertia parameters are not contained as parameters in the neural network architecture instead the network parameters are trained purely by minimizing the data loss defined in (3) with the addition of the initial condition loss i.e

$$\mathcal{L}_{\text{IC}}(\theta) = \left\|\hat{\mathbf{\Omega}}_\theta(0) - \mathbf{\Omega}(0)\right\|^2$$

giving us minimization

$$\theta^* = \min_\theta \left(\mathcal{L}_{\text{data}}(\theta) + \mathcal{L}_{\text{IC}}(\theta)\right)$$

This stage produces a smooth approximation $\hat{\mathbf{\Omega}}_{\theta^*}(t)$ of the observed motion without enforcing any physical constraints.

2.3.2 Stage 2: physics-informed parameter identification

The network parameters θ of Stage 1 are now frozen at θ^*, and the moment of inertia parameters are treated as trainable variables. The goal is to find $\mathbf{I}$ such as to minimize the physics loss

$$\phi^* = \min_{\phi} \mathcal{L}_{\text{phys}}(\phi)$$

where ϕ now represents the weights assigned to the neurons representing the moment of inertia parameters. Since $\hat{\mathbf{\Omega}}_{\theta^*}(t)$ is fixed, this stage reduces to an inverse problem in which the Euler residual is driven to zero by adjusting only the inertia parameters.

This two stage procedure effectively decouples the trajectory approximation from the parameter identification.

3 Methodology

3.1 Two stage neural network for parameters identification

Now that we have established the dynamics involved we move onto identifying the parameters of the problem, namely $l_{11}, l_{22}, l_{33}, l_{21}, l_{31}, l_{32}$, which will involve estimating the model parameters and analyzing the system response. To do this we employ the two stage neural network framework as described above, the network structure of Stage 1 is given in Figure 1.

This neural network is chosen as it allows us to generate a smooth trajectory curve over the entire given time interval even from criticized data of the true trajectory. Further, this network does not require any knowledge of the physics involved as it is just simply mapping time data to a trajectory such as to minimize the difference from the true trajectory. To employ this neural network we require the true trajectory data and to do this we note that the chosen external torque experienced by the body is

$$\boldsymbol{\tau}(t) = \sum_{m=1}^{5} \frac{5}{\sqrt{\pi}} \exp\left(-100(t - t_m)^2\right) \mathbf{J}_m$$

where each $\mathbf{J}_m \in R^3$ is a normalized direction vector and the t_m are a set of sorted random times. This can be thought of as impulses striking the body at times t_m in the

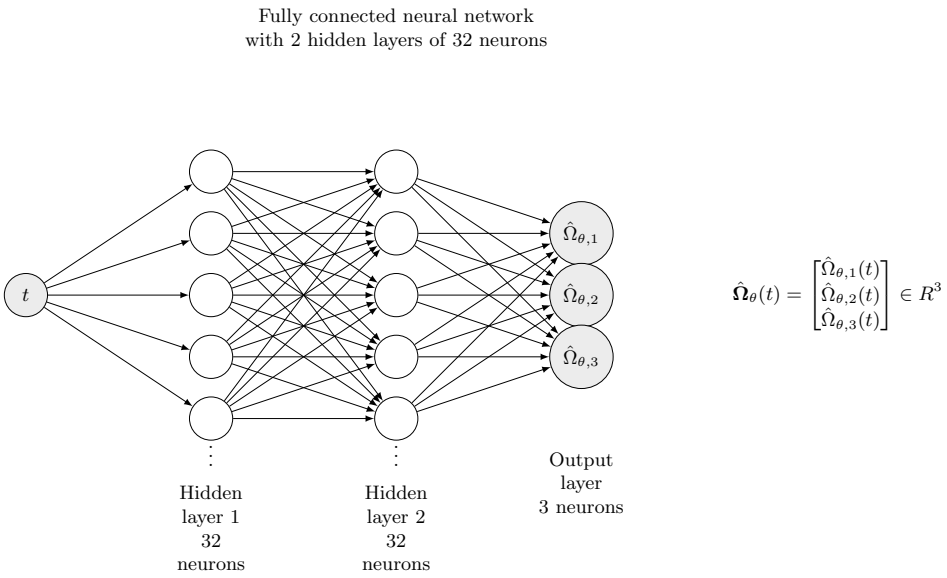


Figure 1: Neural network architecture for Stage 1. The purpose of this network is to generate a smooth trajectory for $\mathbf{\Omega}(t)$ across the time span [0,1]

direction of $\mathbf{J}_m$. With this torque we use the command `solve_ivp(Euler_data, [0,1], [0.8,-1.2,0.5], t_eval=T_eval, args=(tau_func,),rtol=1e-9, atol=1e-12)` of the Numpy module, the `Euler_data` is just a function that returns the time derivative of Eulers equation, `T_eval` is a set of 64 random time points, and the rtol and atol tolerances are chosen to be low enough as to not cause problems in Stage 2. With this, we can actually create our network, the network architecture is again given in Figure 1 with 2 hidden layers of 32 neurons with activations tanh between layers. The network now takes time t, pushes it forward through all the neurons to produce a prediction $\hat{\mathbf{\Omega}}(t)$ and compares it to the true $\mathbf{\Omega}(t)$ using the loss function (3). Automatic differentiation then applies the chain rule through every operation in the network to determine how the loss changes with respect to every weight in every neuron. The optimizer uses these gradients and the chosen learning rate to update each parameter; this slightly adjusts the weight so that next time its closer to the true $\mathbf{\Omega}(t)$. Since the convergence of our parameters in Stage 2 is highly dependent on the convergence of the trajectory here we choose to have the NN work through 1000000 epochs with a custom learning rate that starts of at 1e-2 for the first 5000 epochs, then drops to 1e-3 decaying by half every 10000 epochs until it reaches 100000 epochs and the learning rate is clamped.

With the method of Stage 1 laid out we move onto the surprisingly simple Stage 2.

After freezing the neuron weights in Stage 1 and introducing the moment of inertia parameters as trainable variables as seen in Figure 2. To ensure the PINN doesn't try and look for negative or zero along the diagonal of the inertia tensor we we use the `tf.nn.softplus` command in Tensorflow which causes PINN to see the diagonal elements, I_{ii}, as $I'_{ii} = \log(1 + e^{I_{ii}}) + 10^{-6}$. Next, since the neural network in Stage 1 has already approximated the trajectory we wish not to have the remaining loss in the tra-

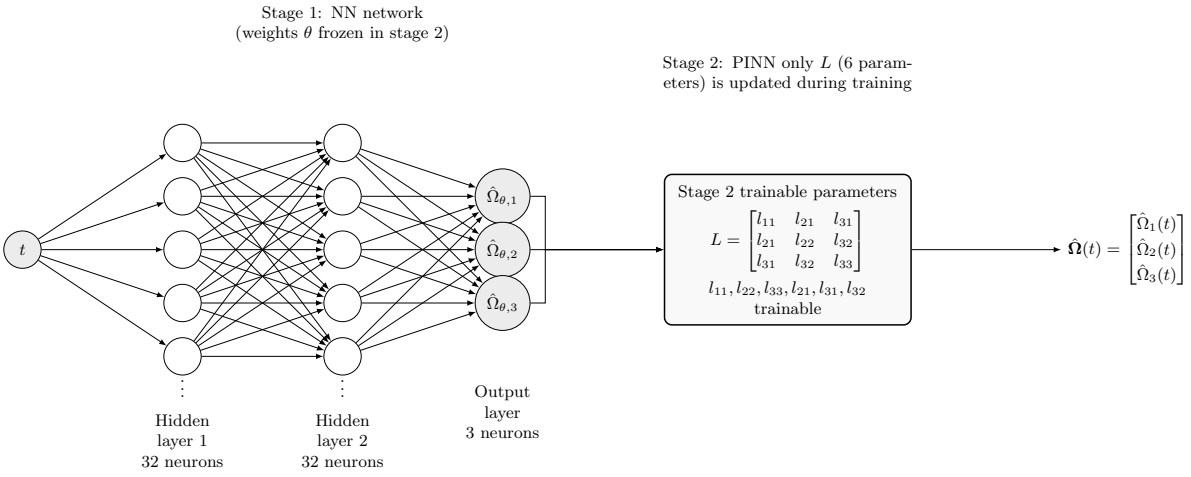


Figure 2: Physics Informed Neural Network architecture for Stage 2. The purpose of this network is to determine the moment of inertia parameters from the smooth learned $\hat{\boldsymbol{\Omega}}_\theta(t)$ across the time span [0,1].

jectory present in the loss calculations for Stage 2, therefore, the loss in Stage 2 is purely the loss given by the Euler residual which is seen in Eq 5 with the torque function defined above. Lastly, the number of epochs for Stage 2 was set to a more moderate 50000 with a different learning rate schedule of 1e-3 for the first 2000 epochs and then reduced to 5e-4 with a decay rate of 0.5 for the remainder of the learning process.

3.2 ChatGPT 5.1 Thinking proposal

The message chosen to send to ChatGPT 5.1 Thinking is the following:

> *I need you to write me a fully working PINN model that will solve the Euler's rigid body equation* $I\frac{d\Omega}{dt} + \Omega \times (I\Omega) = \tau(t)$ *where* Ω *is the angular position of the body and* I *is the moment of inertia matrix over the interval* $[0, 1]$*. The PINN should create synthetic data for* Ω *to learn a smooth trajectory over the time interval and determine the 6 unknown moment of inertia parameters.*

Further, the program must be coded in Python using the Tensorflow package as the main Neural Network package. The external torque experienced on the body is given by [insert torque definition function along with the impulse times t_m and directions $\boldsymbol{J}_k$ providing the seed for accurate comparison]. The true moment of inertia matrix should be

$$I_{true} = \begin{bmatrix} 1 & 0.5 & 0.7 \\ 0.5 & 2 & 0.2 \\ 0.7 & 0.2 & 3 \end{bmatrix}$$

and initial condition Ω_0 = [0.8,-1.2,0.5].

we allowed ourselves to give ChatGPT 5.1 Thinking 5 messages where all we would provide it were the error messages if any or if poor convergence provide the output of the algorithm.

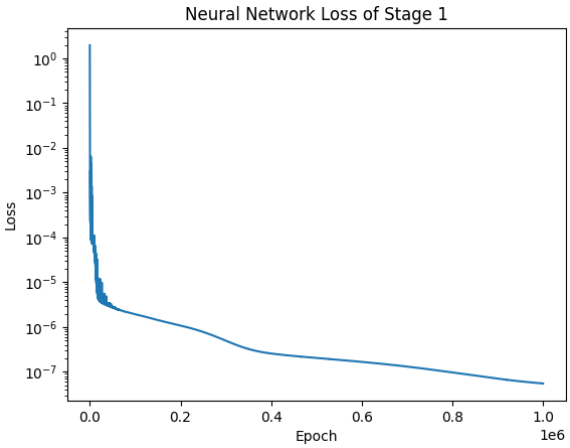
4 Results

4.1 Two stage neural network results

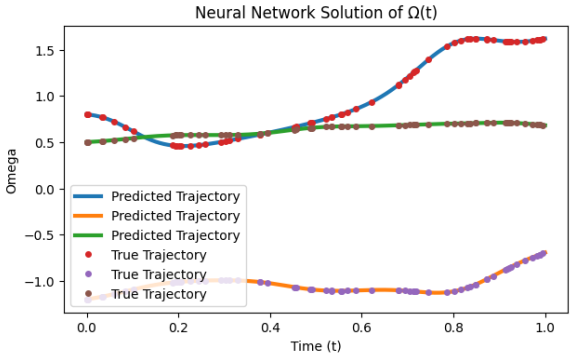
Stage 1 of the two stage PINN framework achieved a loss of 5.5×10^{-8} after 1000000 epochs, the loss curve can be seen on Figure 3(a). The plot of all three trajectories with the true values are also seen in Figure 3(b). With these the loss of Stage 2 is seen in Figure 4 in which the loss of Stage 2 plateaued at a value of 0.00062061 at roughly 11000 epochs.

After the completion of the Stage 2 training we need to reverse the softplus function applied to the diagonal inertia parameters and therefore we compute $I_{ii} = log\left(\exp\left(I'_{ii} - 10^{-6}\right) - 1\right)$ where I'_{ii} are the softplus parameters ensure they remained positive and non negative. Therefore, the moment of inertia tensor obtained from the two stage PINN is

$$\boldsymbol{I}_{learned} = \begin{pmatrix} 1.0021799397161126 & 0.5048770904541016 & 0.7004078030586243 \\ 0.5048770904541016 & 2.0074444757975565 & 0.20027990639209747 \\ 0.7004078030586243 & 0.20027990639209747 & 3.01437672995543 \end{pmatrix}$$



(a) Loss curve of Stage 1 of the PINN in which the NN is approximating the learned $\hat{\mathbf{\Omega}}_\theta(t)$ with the true $\mathbf{\Omega}(t)$ across the time span [0,1].



(b) Predicted trajectory of the body after 1000000 epochs overlaid on the true trajectories.

Figure 3: Plots of the loss per epoch along with the predicted trajectory overlaid on the true trajectories over the time interval [0,1].

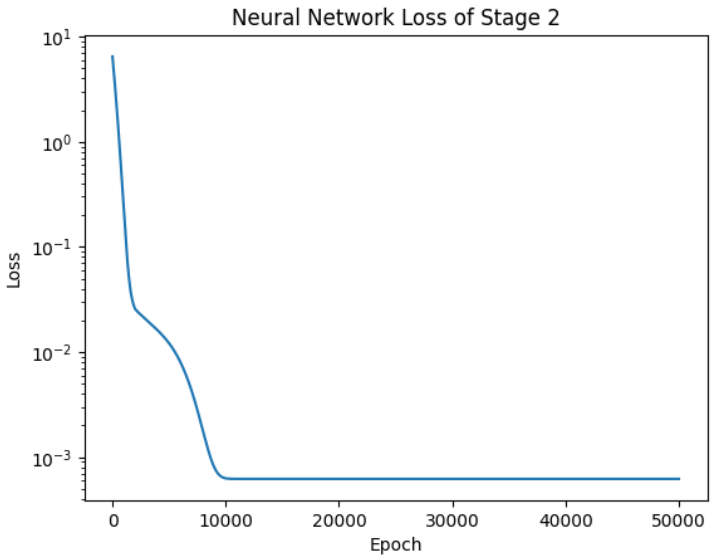


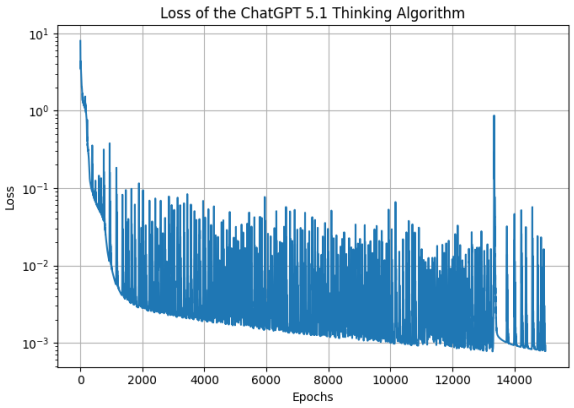
Figure 4: Loss of the PINN in Stage 2. The overall loss plateaued at a value of 0.00062061 at roughly the 10000th epoch.

The external torque profile acting on the body can be seen in Figure 6 of the Appendix (The increases and decreases of Figure 3(b) line up with the impulse times of Figure 6)

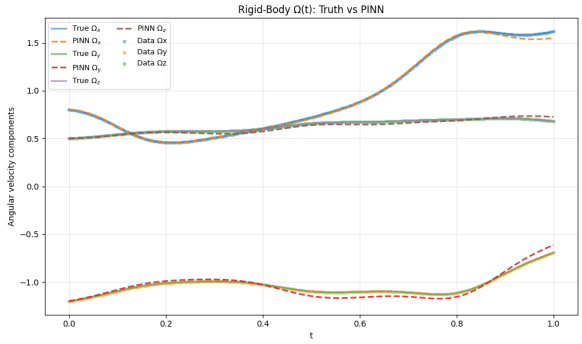
4.2 ChatGPT 5.1 Thinking algorithm results

After 10 messages with the ChatGPT 5.1 Thinking model the 10th message contained a ValueError so the code in message 9 was chosen instead as the representative PINN algorithm generated by ChatGPT 5.1 Thinking. The code in its entirety is under Listing

1 of the Appendix. In order to obtain a plot of the loss during training I altered the code provided be the LLM to include an array that saves the loss at every epoch, this change can be seen preceding the "ADDED BY US" comment in Listing 1 of the Appendix. This was the only aspect of the code altered by us. Thus the loss achieved by LLM after 15000 was 3.8198e-01 with the loss curve present in Figure 5. The predicted trajectory curves estimated by the LLM algorithm can also be seen in



(a) Loss curve of the PINN constructed by ChatGPT 5.1 Thinking over 15000 epochs.



(b) Predicted trajectory of the body as determined by the ChatGPT 5.1 Thinking model after 15000 epochs overlaid on the true trajectories.

Figure 5: Plots of the loss of the PINN over all 15000 epochs and the trajectory curves of the predicted solution of Euler equations determined by the PINN overlaid on the true trajectories.

The moment of inertia tensor after the 15000 epochs as determined by the ChatGPT 5.1 Thinking model came out to be

$$\boldsymbol{I}_{learned,LLM} = \begin{pmatrix} 0.86091 & 0.22983 & 0.54152 \\ 0.22983 & 1.14674 & -0.05997 \\ 0.54152 & -0.05997 & 1.90842 \end{pmatrix}$$

5 Discussion

The error observed in Stage 1 after 1000000 epochs ended up being $\sim 10^{-8}$ which is very good in terms of approximations, however, upon analyzing the loss curve in Figure 3(a) we see that the trend us still decreasing in an almost linear fashion, telling us that the loss has room to be even smaller possibly closer to the order of 10^{-9}. Secondly, given the fact that the loss of Stage 2 dropped to its plateau value by the 10000th epoch we can say that the convergence of the inertia parameters in Stage 2 heavily depend on how well

the approximation of Stage 1 is. Given that, it was absolutely crucial to define a learning rate that would start of large to determine the general loss basin in loss space and then increasingly lower it to increase the resolution of the loss space. This can also be seen by considering that the loss of Stage 1 is only $\sim 10^{-8}$ but the PINN of Stage 2 was only able to find one moment of inertia that would fit the approximated data of Stage 1. Therefore, by increasing the accuracy of Stage 1 (which appears for now to be just increasing the epochs) the inertia parameters can converge even closer to their true values.

Next unfortunately, the PINN algorithm provided by the ChatGPT 5.1 Thinking model did not converge to the right moment of inertia parameters and finished with an overall loss of 0.38198. This could be due to a number of reasons, first by looking at the loss over the 15000 epochs (Figure 5(a)) we notice that almost immediately the loss becomes very noisy meaning that the learning rate was set to high, the NN was stuck in a basin with very large loss relative to its learning rate however the learning rate was low enough to increase resolution at the bottom of this loss basin, next another main reason this PINN did not converge is due to the sheer amount of neurons contained in the network; the network had 3 layers of 64 neurons but only 15000 epochs, when compared to my neural network of 2 layers each with 32 neurons and 1000000 epochs the NN provided by the LLM lacks in epochs and overestimated the amount of neurons required. With all these possible sources for problems the LLM also decided to combine both the learning of the trajectory with the learning of the parameters, do to the size of the network already adding the 6 inertia parameters to the network did not help it terms of training. Unfortunately do the algorithm provided by the LLM not converging we cant really compare computational efficiency however we can state that in the 3 minutes it took the LLM algorithm to reach a loss of 0.38198 our two stage algorithm reached a loss of 0.0000025261. This is of course only comparing the trajectories not the parameters but its worth noting here.

6 Conclusion

In summary, this project explored a physics-informed approach to inertia identification for rigid body dynamics by formulating a two-stage PINN tailored to Euler's equations. The first stage learned a smooth representation of the angular velocity trajectory purely from synthetic data and achieved a loss of 5.5×10^{-8}, while the second stage enforced the rigid

body equations and treated the moments of inertia as trainable parameters, yielding a Stage 2 error of 0.00062061 good convergence behavior towards the true moment of inertia parameters. Situating this method alongside prior PINN applications in civil structures and robotic systems shows that embedding physics into neural network training is an effective strategy for parameter identification in a wide range of mechanical systems. In addition, by implementing the framework both manually and via the ChatGPT 5.1 Thinking model, we gained insight into the current strengths and limitations of LLM-assisted code generation: the generated code was capable of reproducing the overall methodology, but differences in efficiency, organization, and robustness highlight the continued importance of human oversight in specialized scientific computing workflows. The LLM algorithm only achieved a loss of 0.38198 with very poor convergence to the true parameters and a very poor trajectory approximation. Overall, the results suggest that two-stage PINNs along with careful treatment of learning rates are a promising tool for rigid body inertia identification and that LLMs can serve as useful collaborators—though not yet complete replacements—for developing physics-informed machine learning pipelines.

References

[1] M. Mammarella, C. Donati, F. Dabbene, C. Novara, and C. M. Lagoa, "A blended physics-based and black-box identification approach for spacecraft inertia estimation," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*, Milan, Italy, Dec. 2024, pp. 8282–8287.

[2] M. Raissi, P. Perdikaris, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," eng, *Journal of computational physics*, vol. 378, pp. 686–707, 2019, ISSN: 0021-9991.

[3] X. Yang, Z. Zhou, L. Li, and X. Zhang, "Collaborative robot dynamics with physical human–robot interaction and parameter identification with pinn," eng, *Mechanism and machine theory*, vol. 189, pp. 105 439–, 2023, ISSN: 0094-114X.

[4] X.-Y. Guo and S.-E. Fang, "Structural parameter identification using physics-informed neural networks," eng, *Measurement : journal of the International Measurement Confederation*, vol. 220, pp. 113 334–, 2023, ISSN: 0263-2241.

7 Appendix

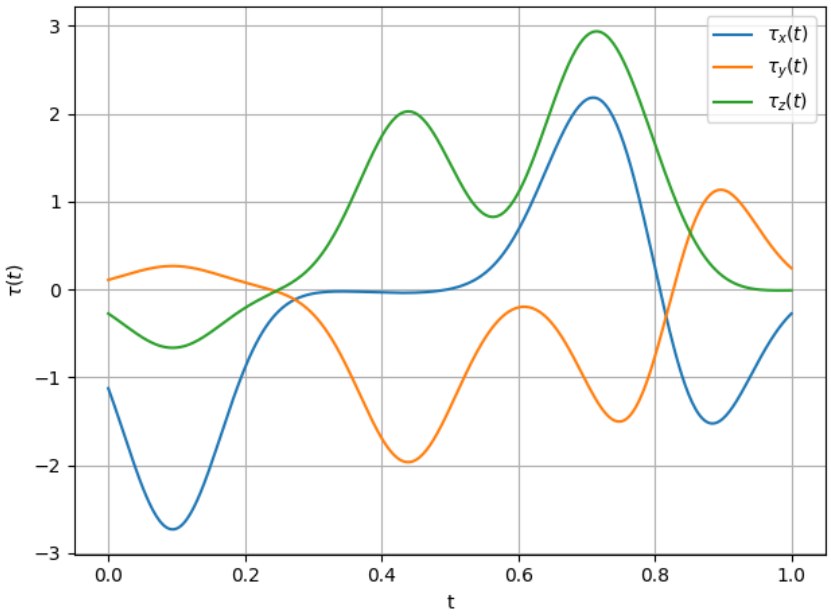


Figure 6: External torque profile acting on the body. The torque in all three axes have been overlaid on the same plot.

...

Listing 1: PINN model for Euler's rigid body equation

```
# PINN for Euler rigid-body dynamics with unknown inertia matrix (
   symmetric 3x3)
# I  d  /dt +       (I    ) =    (t),  t in [0,1]
# Generates synthetic data with the given true I and    (0), then
   trains a PINN to recover    (t) and I.

import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

# ----------------------------
# Problem setup (given)
# ----------------------------
def make_tau_func(t_k, J_k):
    # returns a function tau(t) -> (3,)
    def tau_of_t(t):
        dt = t - t_k                        # shape (N,)
        w = (5/np.sqrt(np.pi)) * np.exp(-(10*dt)**2)   # (N,)
        return (w[:, None] * J_k).sum(axis=0)          # (3,)
    return tau_of_t

rng = np.random.default_rng(42)
N = 5
t_k = np.sort(rng.uniform(0, 1, size=N))
J_raw = rng.normal(size=(N, 3))
J_k = J_raw / np.linalg.norm(J_raw, axis=1, keepdims=True)
tau_func = make_tau_func(t_k, J_k)

I_true = np.array([[1.0, 0.5, 0.7],
                   [0.5, 2.0, 0.2],
                   [0.7, 0.2, 3.0]], dtype=float)
Omega0 = np.array([0.8, -1.2, 0.5], dtype=float)
```

```
# ----------------------------
# Helper: deterministic RK4 integrator to synthesize   (t) data
# ----------------------------
def euler_rhs(t, omega, I, tau_fun):
    # I  d  /dt +       (I   ) =   (t)  ->  d  /dt = I^{-1} [   (t)
       -       (I   ) ]
    Io = I @ omega
    rhs = np.linalg.solve(I, tau_fun(t) - np.cross(omega, Io))
    return rhs

def rk4_trajectory(t0, t1, dt, omega0, I, tau_fun):
    ts = np.arange(t0, t1 + 1e-12, dt)
    w = np.zeros((ts.size, 3), dtype=float)
    w[0] = omega0
    for i in range(ts.size - 1):
        t = ts[i]
        h = dt
        y = w[i]
        k1 = euler_rhs(t,          y,                I, tau_fun)
        k2 = euler_rhs(t + h/2.0,  y + h*k1/2.0,     I, tau_fun)
        k3 = euler_rhs(t + h/2.0,  y + h*k2/2.0,     I, tau_fun)
        k4 = euler_rhs(t + h,      y + h*k3,         I, tau_fun)
        w[i+1] = y + (h/6.0)*(k1 + 2*k2 + 2*k3 + k4)
    return ts, w

# Synthesize "ground truth"   (t) and pick training samples
t0, t1 = 0.0, 1.0
dt_truth = 1.0/4000.0  # fine integration for smooth "true"
   reference
t_truth, omega_truth = rk4_trajectory(t0, t1, dt_truth, Omega0,
   I_true, tau_func)

# Select observed data points (you can tweak count/noise)
```

```
rng = np.random.default_rng(1)
n_data = 200
idx = np.linspace(0, t_truth.size - 1, n_data).astype(int)
t_data = t_truth[idx]
omega_data = omega_truth[idx].copy()
noise_std = 0.00  # set small noise if desired (e.g., 0.01)
omega_data += rng.normal(scale=noise_std, size=omega_data.shape)

# ----------------------------
# TensorFlow PINN
# ----------------------------
tf.keras.utils.set_random_seed(1234)
tf.keras.backend.set_floatx('float32')

# Build a small MLP: t -> R^3 (  (t))
def make_model():
    return tf.keras.Sequential([
        tf.keras.layers.InputLayer(shape=(1,)),
        tf.keras.layers.Dense(64, activation='tanh'),
        tf.keras.layers.Dense(64, activation='tanh'),
        tf.keras.layers.Dense(64, activation='tanh'),
        tf.keras.layers.Dense(3,  activation=None),
    ])

model = make_model()

# Trainable symmetric inertia matrix with softplus on diagonal to
   keep it positive
# Parameters: a,b,c,d,e,f correspond to:
# [[a, b, c],
#  [b, d, e],
#  [c, e, f]]  with a,d,f = softplus(raw) + eps
raw_a = tf.Variable(0.7, dtype=tf.float32)
raw_d = tf.Variable(0.7, dtype=tf.float32)
```

```
raw_f = tf.Variable(0.7, dtype=tf.float32)
b = tf.Variable(0.1, dtype=tf.float32)
c = tf.Variable(0.1, dtype=tf.float32)
e = tf.Variable(0.1, dtype=tf.float32)

def softplus_pos(x):  # strictly positive
    return tf.nn.softplus(x) + 1e-3

def inertia_matrix():
    a = softplus_pos(raw_a)
    d = softplus_pos(raw_d)
    f = softplus_pos(raw_f)
    I = tf.stack([
        tf.stack([a, b, c], axis=0),
        tf.stack([b, d, e], axis=0),
        tf.stack([c, e, f], axis=0),
    ], axis=0)  # (3,3)
    return I

# Precompute torque parameters into TF constants for a vectorized
     (t)
t_k_tf = tf.constant(t_k.reshape(-1, 1), dtype=tf.float32)   # (N
   ,1)
J_k_tf = tf.constant(J_k, dtype=tf.float32)                  # (N
   ,3)

def tau_tf(t_batch):
    """
    Vectorized   (t) for batch times.
    t_batch: (B,1)
    returns: (B,3)
    """
    dt = t_batch - tf.transpose(t_k_tf)             # (B,N)
    w = (5.0/np.sqrt(np.pi)) * tf.exp(-(10.0*dt)**2)  # (B,N)
```

```
    tau = tf.matmul(w, J_k_tf)                          # (B,3)
    return tau

# Cross product helper for batch vectors
def batch_cross(a, b):
    # a,b: (B,3)
    a1,a2,a3 = a[:,0], a[:,1], a[:,2]
    b1,b2,b3 = b[:,0], b[:,1], b[:,2]
    c1 = a2*b3 - a3*b2
    c2 = a3*b1 - a1*b3
    c3 = a1*b2 - a2*b1
    return tf.stack([c1,c2,c3], axis=1)  # (B,3)

# Physics residual r(t) = I  d  /dt +       (I    ) -   (t)
@tf.function
def physics_residual(t_c):
    I = inertia_matrix()                 # (3,3)
    with tf.GradientTape() as tape:
        tape.watch(t_c)                  # t_c: (B,1)
        omega = model(t_c)               # (B,3)
    # Correct: (B,3,1) -> squeeze last axis -> (B,3)
    domega_dt = tf.squeeze(tape.batch_jacobian(omega, t_c), axis
       =-1)  # (B,3)

    # I  d  /dt -> (B,3)
    Idomega = tf.linalg.matvec(I, domega_dt)   # (B,3)
    Io = tf.linalg.matvec(I, omega)            # (B,3)
    cross_term = batch_cross(omega, Io)        # (B,3)
    res = Idomega + cross_term - tau_tf(t_c)   # (B,3)
    return res

# Losses
w_phys = 1.0
w_data = 1.0
```

```
w_ic   = 5.0

t_data_tf = tf.constant(t_data.reshape(-1,1), dtype=tf.float32)
       # (Nd,1)
omega_data_tf = tf.constant(omega_data, dtype=tf.float32)
             # (Nd,3)

# Collocation points for physics residual
n_colloc = 1000
t_c_tf = tf.constant(rng.uniform(0.0, 1.0, size=(n_colloc,1)).
   astype(np.float32))

# Initial condition point
t0_tf = tf.constant([[0.0]], dtype=tf.float32)
omega0_tf = tf.constant(Omega0.reshape(1,3), dtype=tf.float32)

optimizer = tf.keras.optimizers.Adam(learning_rate=5e-3)

@tf.function
def train_step():
    with tf.GradientTape(persistent=True) as tape:
        # Data loss
        omega_pred_data = model(t_data_tf)
        L_data = tf.reduce_mean((omega_pred_data - omega_data_tf)
           **2)

        # Physics loss
        res = physics_residual(t_c_tf)
        L_phys = tf.reduce_mean(tf.reduce_sum(res**2, axis=1))

        # Initial condition loss
        omega0_pred = model(t0_tf)
        L_ic = tf.reduce_mean((omega0_pred - omega0_tf)**2)
```

```
        L_total = w_data*L_data + w_phys*L_phys + w_ic*L_ic

    vars_all = model.trainable_variables + [raw_a, raw_d, raw_f, b,
        c, e]
    grads = tape.gradient(L_total, vars_all)
    optimizer.apply_gradients(zip(grads, vars_all))
    return L_total, L_data, L_phys, L_ic

# ----------------------------
# Training
# ----------------------------
max_epochs = 15000
print_every = 500

loss=[] ### ADDED BY US

for ep in range(1, max_epochs+1):
    Ltot, Ld, Lp, Lic = train_step()
    loss.append(Ltot.numpy()) ### ADDED BY US
    if ep % print_every == 0 or ep == 1:
        I_est = inertia_matrix().numpy()
        print(f"Epoch {ep:5d} | L_tot={Ltot.numpy():.4e} | L_data={
           Ld.numpy():.4e} | "
              f"L_phys={Lp.numpy():.4e} | L_ic={Lic.numpy():.4e}")
        print("  I_est    \n", I_est)

# ----------------------------
# Evaluation & reports
# ----------------------------
I_learned = inertia_matrix().numpy()
print("\nTrue inertia:\n", I_true)
print("\nLearned inertia:\n", I_learned)

# Predict on a fine grid and compare with truth
```

```
t_plot = np.linspace(0, 1, 500, dtype=np.float32).reshape(-1,1)
omega_pred = model(tf.constant(t_plot)).numpy()

# Relative error on inertia entries
rel_err_I = np.linalg.norm(I_learned - I_true) / np.linalg.norm(
   I_true)
print(f"\nRelative error ||I_est - I_true|| / ||I_true|| = {
   rel_err_I:.4e}")

# ----------------------------
# Plots
# ----------------------------
plt.figure(figsize=(10,6))
labels = [r'$\Omega_x$', r'$\Omega_y$', r'$\Omega_z$']
for i in range(3):
    plt.plot(t_truth, omega_truth[:,i], lw=2, label=f'True {labels[
       i]}', alpha=0.6)
    plt.plot(t_plot[:,0], omega_pred[:,i], '--', lw=2, label=f'PINN
        {labels[i]}', alpha=0.9)
plt.scatter(t_data, omega_data[:,0], s=12, alpha=0.4, label='Data
    x ')
plt.scatter(t_data, omega_data[:,1], s=12, alpha=0.4, label='Data
    y ')
plt.scatter(t_data, omega_data[:,2], s=12, alpha=0.4, label='Data
    z ')
plt.xlabel('t')
plt.ylabel('Angular velocity components')
plt.title('Rigid-Body   (t): Truth vs PINN')
plt.legend(ncol=2, fontsize=9)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Also print final estimate nicely
```

```
def fmt_mat(M):
    return "\n".join(["[" + "  ".join(f"{v: .5f}" for v in row) +
       "]" for row in M])

print("\nFinal learned inertia matrix I:")
print(fmt_mat(I_learned))
```